Python 运算符核心笔记与避坑指南

> **核心心法**：运算符是让程序拥有“判断力”和“计算力”的基石。掌握它们，你就能让电脑按照你的逻辑去处理复杂任务。

---

1. 算术运算符（数学计算）

负责基础的数学运算。

- `+ - * /`：加、减、乘、除。**注意：`/` 除法的结果永远是浮点数（小数）。**
- `//`：地板除（向下取整，只留整数部分）。
- `%`：取模（求余数，常用于判断奇偶数，如 `x % 2 == 0` 就是偶数）。
- `**`：幂运算（如 `2 ** 3` 等于 8）。

---

2. 赋值运算符（装箱子）

负责把右边的计算结果，存入左边的变量中。

- `=`：基础赋值。 **（注意：单个等号是赋值，不是比较！）**
- `+= -= *= /=`：快捷赋值。例如 `a += 1` 等价于 `a = a + 1`。

---

3. 比较运算符（当裁判）

负责比较两边的值，结果永远是布尔值：`True`（真）或 `False`（假）。

- `==`：等于。 **（注意：两个等号才是判断是否相等！）**
- `!=`：不等于。
- `> < >= <=`：大于、小于、大于等于、小于等于。

---

4. 逻辑运算符（多重条件）

负责组合多个判断条件，是写出复杂业务逻辑的核心。

运算符	核心口诀	偷懒规则（短路逻辑）
`and` (与)	**一假必假**（必须全为 True）	只要遇到第一个 `False`，后面的条件直接不看，立刻返回 `False`。
`or` (或)	**一真必真**（只要有一个 True 即可）	只要遇到第一个 `True`，后面的条件直接不看，立刻返回 `True`。
`not` (非)	**颠倒黑白**（True 变 False，反之亦然）	常用于反向判断或检查“空状态”。

---

5. 成员运算符（查户口）

专门用于检查某个元素是否存在于容器（如字符串、列表、集合、字典）中。

- `in`：如果元素在容器里，返回 `True`。
- `not in`：如果元素**不**在容器里，返回 `True`。（常用于黑名单拦截）。

🌟 进阶神技：Pythonic 的“空值判断”

在 Python 中，所有数据天生自带真假属性。**只要是“空的”或“零”，天生就是 `False`。**

- **自带 `False` 的常见空值：** `""`（空字符串）、`[]`（空列表）、`set()`（空集合）、`{}`（空字典）、`0`、`None`。
- **秒杀技巧：** 判断用户是否没输入、或者列表是否为空时，**不要写** `if len(name) == 0:`，**直接写** `if not name:`。优雅又高效！

⚠️ 终极避坑指南（血泪教训总结）

1. `input()` 的跨频道聊天陷阱

- `input()` 获取到的内容**永远是字符串（文字）**。
- 如果你要拿它和数字比大小，**必须加 `int()`**：`age = int(input("年龄:"))`
- 如果你要拿它做逻辑判断，**直接和文字比**：`if answer == "是":`，**绝对不要**写成 `if answer == True:`。

2. 拼写与大小写

- Python 对大小写极其敏感。逻辑真假必须首字母大写：`True` 和 `False`。写成 `Ture` 或 `true` 都会报错或导致逻辑失效。
- 处理用户输入时，可以用 `.strip().lower()` 消除空格和大小写差异，让你的程序更“抗揍”。

3. 优先级拿不准？加括号！

- 当 `and`、`or`、`not` 混在一起时，`not` 优先级最高。
- 为了防止电脑算错，也为了自己能看懂，**多用 `()` 把条件分组**。例如：`if (a > 5) and (not b):`。